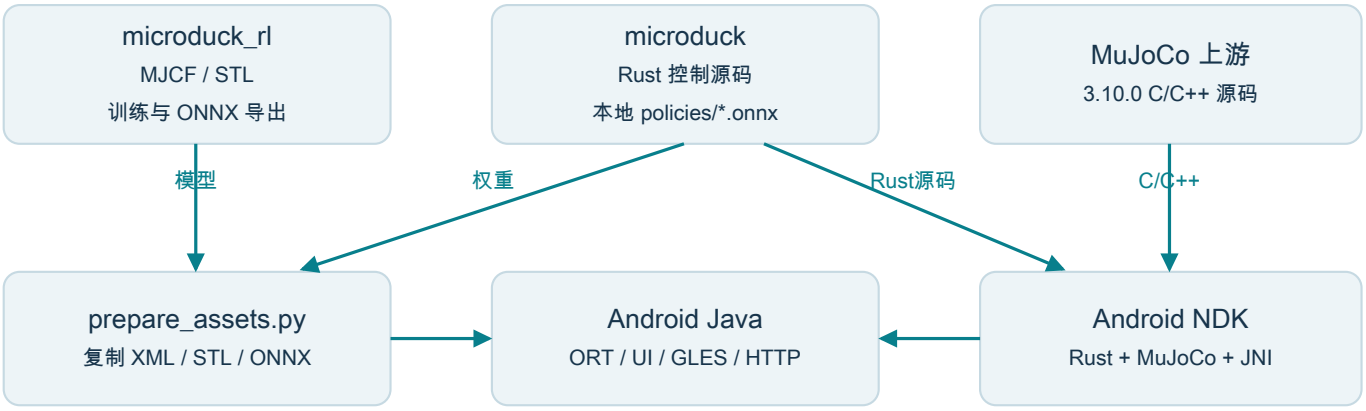
Android Microduck
方案、架构与编译复用

从资源来源到 APK，从策略输入到机器人相机

核对日期：2026-10-05
当前 Android：0.10.0-m10 / versionCode 18
三项目边界、构建链、关键代码、运行方法与验收证据。



M10：Rust核心进入APK；Python训练环境与桌面daemon留在主机

A/ = android-sim/；R/ = Pollen/microduck/；T/ = Pollen/microduck_rl/。
全文基于本地源码；示例命令没有在本轮执行。用户继续使用 Android Studio 构建和运行。

目录

01 方案定位与阅读结论 4
02 三个项目的职责与复用类型 5
03 Android构建链与产物总览 6
04 Gradle如何触发资源和原生编译 7
05 MuJoCo如何在NDK中编译 8
06 同为ARM64：哪些能直接复用 9
07 依赖清单：按使用阶段区分 10
08 CAD、JSON、MJCF与Android模型收集 11
09 资源收集与手机资源校验的关键代码 12
10 ONNX的来源、导出与策略校验 13
11 APK组成与应用启动顺序 14
12 运行时技术架构与线程所有权 15
13 控制周期与JNI物理接口 16
14 61D观测与14D动作：跨仓库接口契约 17
15 推理、动作滤波与两种执行器 18
16 BAM执行器：移植方案与关键公式 19
17 训练环境与Android物理的剩余差异 21
18 APK内嵌Rust核心：复用范围 22
19 Rust编译与JNI链接的关键代码 24
20 Rust与Android ORT：为什么保留混合链路 25
21 SimIo身体服务：桌面原版控制手机 26
22 两种Rust模式的使用命令 27
23 CAD渲染与机器人相机架构 28
24 浏览器控制、状态与视频传输 30
25 单次前滚翻与当前抬头问题 31
26 与桌面duck-sim的编译/运行关系 32
27 训练插件与新策略接入流程 33
28 新机器恢复工程与Studio编译 34
29 运行使用与只读排查命令 35
30 变更如何传到Android：重建关系表 36
31 检查证据与当前验收边界 37
32 常见构建与运行问题定位 38
33 阶段性交付与项目价值 39

34 后续方向与优先顺序 40

35 开发方法与接续阅读 41

36 附录：源码路径与证据索引 42

01 方案定位与阅读结论

M10已完成功能闭环，并获用户实机确认。

当前Android应用在手机内部运行MuJoCo物理、BAM执行器、Rust控制核心、Android ONNX Runtime和独立机器人相机。另一台Android可通过浏览器控制并观看WebRTC/JPEG视频。手机本地站立、步行、整圈前滚翻不再需要电脑或USB。

构建结论：microduck_rl提供XML/STL，microduck目录提供现成ONNX与直接编译复用的Rust控制源码。Gradle驱动资源收集、Java/AAR与CMake；CMake调用Cargo构建Android ARM64静态库，并将Rust与JNI链接进libmicroduck_m1.so。MuJoCo从固定C/C++源码另编为libmujoco.so，ORT与WebRTC来自Android AAR。

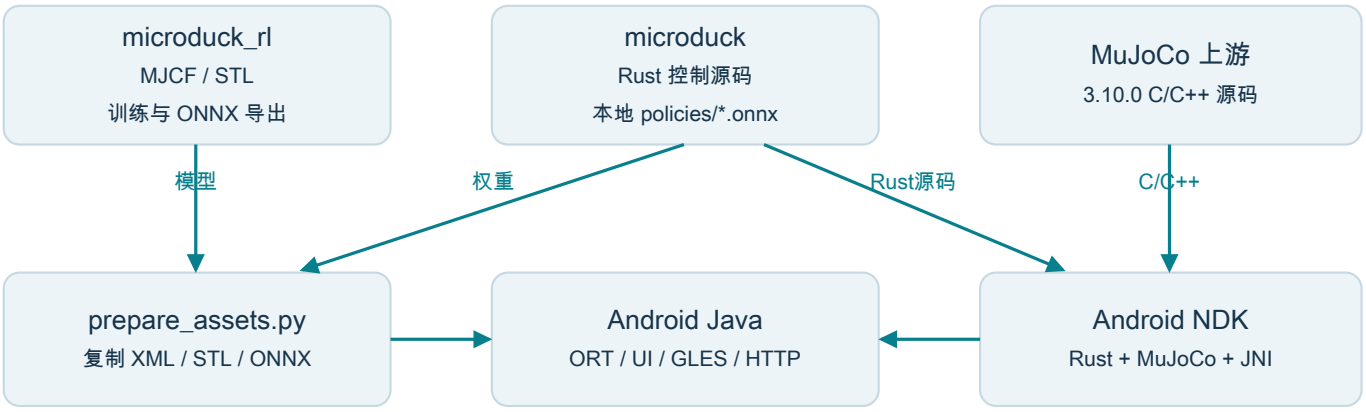
问题	当前实现
是否需要Cargo？	M10构建需要Rust/Cargo和aarch64-linux-android标准库；Studio自动调用，无需先手工编译桌面robotd。
是否需要uv sync或训练？	已有XML/STL和三份ONNX即可打包；Python标准库负责资源脚本，训练环境只用于训练与主机复演。
是否需要重新导出CAD？	读取现成MJCF/XML/STL；模型制作与Android构建是两步。
Rust是否已在APK里？	已静态链接控制核心；完整robotd进程、硬件总线与全部服务没有内嵌。
ONNX是否要改成Rust推理？	当前Rust经JNI调用现有Android ORT。改为Rust绑定仍通常调用ORT引擎，现阶段无需更换已验证后端。

源码依据：A/app/build.gradle:11；A/app/src/main/cpp/CMakeLists.txt:62；A/rust/control-core/src/lib.rs:4。

用户已确认M10实机验证通过，见 A/verification/user-M10-feedback.json:1。这是用户功能确认，不附加推断codec、速度跟踪、温升或长期稳定性结论。本次只更新文档；没有新增APK构建、训练或设备操作。

02 三个项目的职责与复用类型

资源输入、源码编译和运行服务的依赖分别列明。



M10：Rust核心进入APK；Python训练环境与桌面daemon留在主机

M10既读取microduck资源，也直接编译其Rust控制源码。

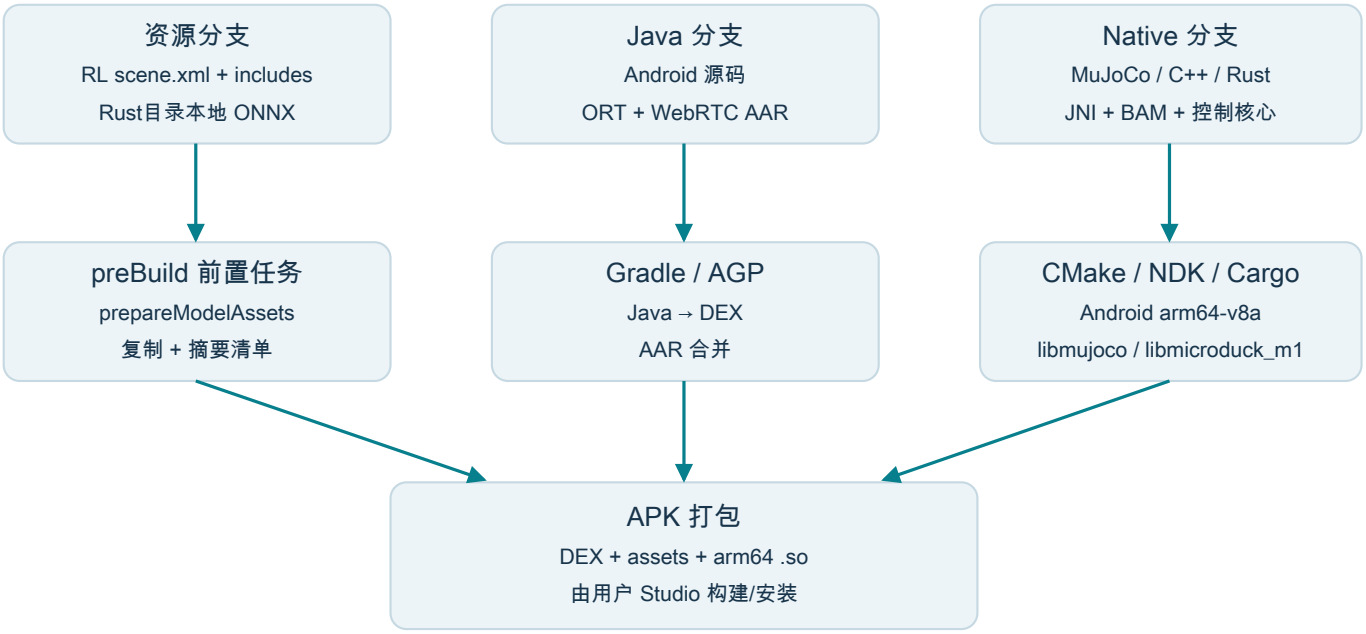
项目	原本职责	Android实际使用
microduck	Rust robotd、控制核心、媒体、板端部署与硬件接口	三份现成ONNX；model/imu/io/obs/safety及Controller源码；params/proto/kinematics path依赖。
microduck_rl	mjlab插件、PPO训练、BAM、模型生成与CPU复演	复制现成XML/STL；采用HOME/关节/相机约定；BAM公式另以C++移植。
android-sim	Java界面/线程/网络/ORT，Rust适配层，C++身体与GLES	组织APK内闭环、浏览器视频和桌面SimIo接管。
MuJoCo上游	通用C/C++物理引擎	固定3.10.0源码，经NDK编译为Android共享库。

直接复用入口：A/rust/control-core/src/lib.rs:6；A/rust/control-core/Cargo.toml:14；来源锁A/RUST_CORE.lock.json:1。训练插件入口 T/pyproject.toml:40。

相同权重和模型不保证相同动作：执行器、延迟、初始化、滤波与策略交接都会影响闭环。M10已复用控制源码，但完整训练环境与完整daemon仍有明确边界。

03 Android构建链与产物总览

资源、Java/AAR、C++与Rust最终汇合为APK。



三条分支汇合；不承诺它们在 Gradle 中固定先后或固定并行

这是逻辑依赖图；Java与原生任务的具体调度由Gradle决定。

构建分支	输入	输出
资源	scene.xml/robot_groundcontact.xml/附加XML/STL；三份ONNX	assets/model和assets/policies的副本与摘要清单
Java	Android Java源码、Android SDK、ORT与WebRTC Android AAR	DEX；AAR的Java/native依赖合并进APK
Native	MuJoCo/C++ JNI/BAM；Rust控制源码与适配层；NDK/Cargo	libmujoco.so、libmicroduck_m1.so，ABI=arm64-v8a
打包	Manifest、DEX、assets、共享库	APK内含静态链接Rust核心；不附带训练.venv或桌面daemon

配置入口：A/app/build.gradle:21；A/app/build.gradle:40。

04 Gradle如何触发资源和原生编译

关键代码：app/build.gradle。

源码：A/app/build.gradle:30

```
30 dependencies {
 31 implementation 'com.microsoft.onnxruntime:onnxruntime-android:1.20.0'
 32 implementation 'io.getstream:stream-webrtc-android:1.3.9'
 33 }
 34
 35 // Rebuild the MJCF dependency bundle from the sibling training repository before packaging.
 36 tasks.register('prepareModelAssets', Exec) {
 37 workingDir rootProject.projectDir
 38 commandLine 'python3', file('../scripts/prepare_assets.py').absolutePath
 39 }
```

注解①：ONNX Runtime以implementation依赖加入，Gradle从Maven仓库解析AAR。AAR提供Java推理API及对应Android native库，工程没有在CMake中从源码构建ORT。

注解②：Stream WebRTC 1.3.9同样以预编译AAR加入；MediaCodec编码及WebRTC传输来自Android SDK/AAR，不从Pollen仓库或GStreamer编译。固定来源和摘要见VIDEO_DEPENDENCIES.lock.json。注解③：prepareModelAssets的工作目录是Android根目录；脚本路径由app模块向上定位到scripts。preBuild依赖该Exec任务，因此正常Studio构建会执行资源收集，不要求用户先手工复制assets。

源码：A/app/build.gradle:12

```
12 ndk { abiFilters 'arm64-v8a' }
 13 externalNativeBuild {
 14 cmake {
 15 arguments '-DANDROID_STL=c++_static', '-DFETCHCONTENT_UPDATES_DISCONNECTED=ON'
 16 targets 'microduck_m1'
 17 }
 18 }
 19 }
 20 externalNativeBuild {
 21 cmake { path file('src/main/cpp/CMakeLists.txt'); version '3.30.3' }
 22 }
```

注解③：abiFilters只打arm64-v8a；externalNativeBuild只请求microduck_m1目标，它通过target_link_libraries带入MuJoCo和Rust静态库。ANDROID_STL=c++_static指定应用原生构建所用C++运行库；不能据此推断ORT AAR内部如何构建。

AGP插件版本与仓库设置：A/build.gradle:3、A/settings.gradle:3；Gradle Wrapper版本：A/gradle/wrapper/gradle-wrapper.properties:3。

UPSTREAM.lock.json是来源记录；prepare_assets.py不会读取它来自动校验上游Git提交或阻止模型资源变化。当前每次复制重新计算manifest，固定基线需要同时检查来源记录、实际文件摘要与smoke。

05 MuJoCo如何在NDK中编译

Android复用的是引擎源码，不是桌面Python包的二进制。

源码：A/app/src/main/cpp/CMakeLists.txt:7

```
7 set(MUJOCO_SOURCE_DIR "${M1_PROJECT_ROOT}/.native-cache/mujoco-3.10.0" CACHE PATH "Pinned MuJoCo source")
 8 find_package(Git REQUIRED)
 9 if(NOT EXISTS "${MUJOCO_SOURCE_DIR}/CMakeLists.txt")
 10 get_filename_component(MUJOCO_CACHE_PARENT "${MUJOCO_SOURCE_DIR}" DIRECTORY)
 11 file(MAKE_DIRECTORY "${MUJOCO_CACHE_PARENT}")
 12 execute_process(COMMAND "${GIT_EXECUTABLE}" clone --depth 1 --branch 3.10.0
 13 https://github.com/google-deepmind/mujoco.git "${MUJOCO_SOURCE_DIR}"
 14 RESULT_VARIABLE CLONE_RESULT)
 15 if(NOT CLONE_RESULT EQUAL 0)
 16 message(FATAL_ERROR "MuJoCo download failed; restore network access and retry Gradle Sync")
 17 endif()
```

缺少.native-cache/mujoco-3.10.0时，CMake克隆固定3.10.0标签。随后检查HEAD必须是28009f9105cd92784b7b0b30c0605a5e29107a77；这里的提交检查是真正的构建拒绝条件。首次配置需要网络和Git。

源码：A/app/src/main/cpp/CMakeLists.txt:52

```
52 add_subdirectory("${MUJOCO_SOURCE_DIR}" mujoco-build EXCLUDE_FROM_ALL)
 53 # Bionic provides localtime_r; upstream 3.10 requires an explicit POSIX feature macro.
 54 target_compile_definitions(mujoco PRIVATE _POSIX_C_SOURCE=200809L)
 55 # Android packages .so files; keep the bundled library SONAME unversioned.
 56 set_target_properties(mujoco PROPERTIES VERSION "" SOVERSION "")
```

注解①：add_subdirectory把MuJoCo C/C++工程纳入当前Android工具链；libmicroduck_m1链接mujoco、Rust静态库和Android系统库。Python wheel里的macOS共享库不能直接作为Android arm64库使用。

注解②：关闭MuJoCo示例、simulate、测试及Python工具，绘制由Android GLES承担；CMake依赖的底层库按MuJoCo自己的固定版本获取。引擎不等同于桌面MuJoCo GUI。

注解③：JNI和MuJoCo链接设置16KB ELF段对齐，但整个APK仍受ORT AAR影响；当前ORT 1.20.0 native库为4KB对齐，已连接设备为4KB页。当前组合的16KB页设备兼容性尚未解决。依据 A/README.md:45。

补丁：A/patches/mujoco-3.10.0-ndk25.patch:1把XML解析的string_view迭代器构造改为指针+长度，适配NDK25 libc++。CMake先反向检查是否已有补丁，再检查并应用，避免重复修改；见 A/app/src/main/cpp/CMakeLists.txt:26。

06 同为ARM64：哪些能直接复用

CPU架构相同，不代表操作系统、libc与二进制ABI相同。

对象	当前复用方式	必要检查
XML / STL	作为数据文件复制，手机MuJoCo重新解析为内存模型	相对路径、版本、惯性/碰撞、相机及关节语义
ONNX	作为网络数据复制，手机ORT Android重新建立Session	支持的算子/模型格式、normalizer、61D/14D与关节顺序、smoke
Python源码/.venv	当前只在主机训练/复演或提供桌面模拟身体	没有打入APK；桌面wheel的原生库不是Android库
Rust板端daemon	完整daemon二进制不复用；控制源码经Cargo重新编为Android静态库	板端目标为Linux GNU ABI；并非Android NDK目标
MuJoCo C/C++源码	经Android NDK工具链重新编译	NDK兼容补丁、API/ABI、链接依赖、版本及页大小
ORT Android AAR	使用厂商为Android构建的Java/native包	其版本/ABI/系统兼容性；不是复用板端ORT安装文件

microduck的cargo board别名明确使用zigbuild --target aarch64-unknown-linux-gnu.2.31；源码注释指出Apple Silicon aarch64-apple-darwin与板端同CPU却不同OS/libc/ABI。Android当前目标是NDK arm64-v8a，源码检查使用aarch64-linux-android28工具链目标。因此不能把板端可执行文件或桌面Python共享库直接放到APK当Android native库。

证据：R/.cargo/config.toml:24、A/scripts/check_source.py:105。真实板端dev-push是Linux daemon构建、打包与部署流程，不是Studio的APK构建或安装路径。

三套ORT也独立管理：Rust工作区声明板端最低1.23、安装目标1.28.0；RL pyproject声明Python ORT≥1.24.4；Android固定AAR1.20.0。它们不是同一份共享库。现有策略已通过跨版本smoke，不代表未来任意新模型都能由旧Android ORT执行。版本依据 R/Cargo.toml:31、T/pyproject.toml、A/app/build.gradle。

M10已建立aarch64-linux-android与JNI/FFI编译链。Rust控制源码直接复用，桌面加载器由Android推理适配器替代；完整daemon的服务/硬件驱动仍留在桌面或板端。依据 A/scripts/build_rust_core.py:28。

07 依赖清单：按使用阶段区分

打包依赖、运行依赖与主机对照工具应分开安装。

阶段	依赖/版本	用途与依据
Android构建	JBR17.0.14 / Gradle7.5 / AGP7.2.2	运行Gradle的JDK为17，Java源码/字节码兼容级别11；A/app/build.gradle:24
Android构建	SDK34 / NDK25.1.8937393 / CMake3.30.3	compile/target=34、min=28；交叉编译arm64；A/app/build.gradle:5
Rust构建	Rust/Cargo；Android ARM64标准库	本机验证1.98.1；CMake调用Cargo并校验源文件锁；A/RUST_CORE.lock.json:6
资源收集	Python3.9+，标准库；Git	Path.is_relative_to等用于复制/摘要；不导入torch/mjlab；A/scripts/prepare_assets.py:3
Android运行	MuJoCo3.10.0 / ORT Android1.20.0	原生物理与Java CPU推理；A/app/build.gradle:31
Android运行	Stream WebRTC1.3.9 / GLES3 / JPEG / Socket	SDK平台能力；Manifest要求GLES3及INTERNET；A/app/src/main/AndroidManifest.xml:3
主机对照	mujoco / numpy / onnxruntime / 可选org.json JAR	Python预演和Mac Java协议检查；不作为训练依赖打包进APK；A/scripts/check_source.py:1
训练端	Python3.12 / mjlab1.3.0 / warp-lang1.12.0 / torch2.9.1 / BAM	pyproject当前声明；CUDA训练条件与Android无关；T/pyproject.toml:15
桌面Rust仿真	Cargo/Rust、Python身体端；相机需GStreamer	scripts/duck-sim启动真实Rust daemon与Python身体；Android不调用这条链。

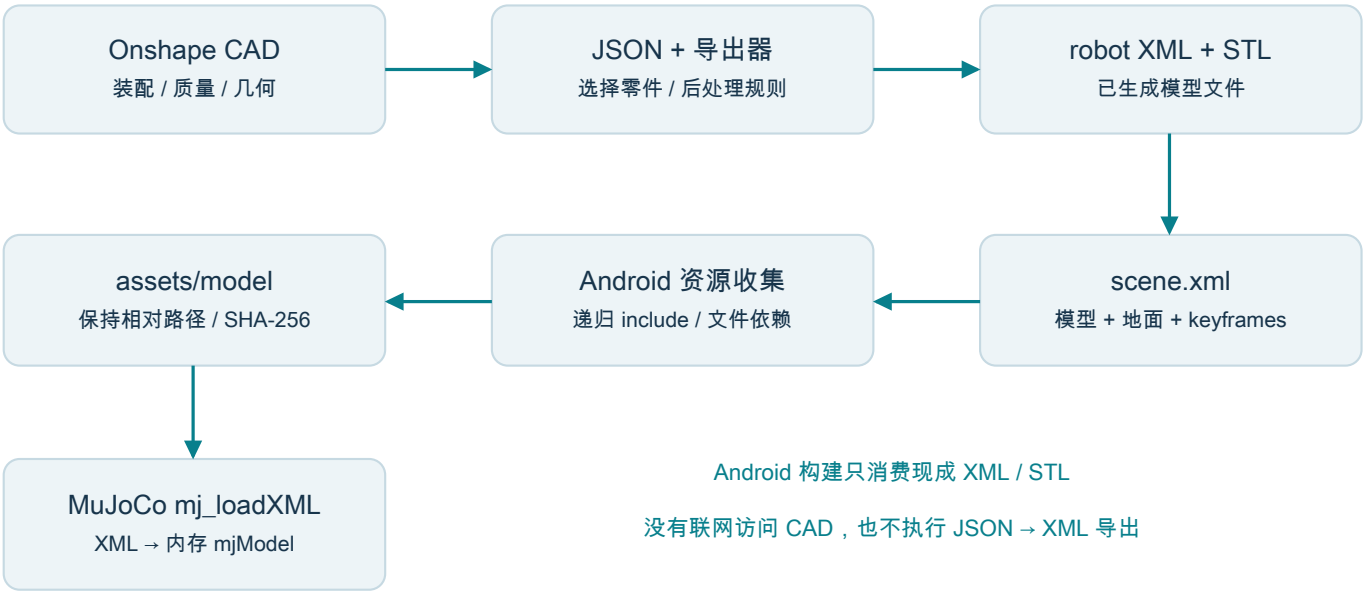
MuJoCo当前缓存优先复用ccd、lodepng、marchingcubecpp、miniz、qhull、tinyobjloader、tinyxml2源码；缺失时由上游FetchContent按固定提交下载。见 A/app/src/main/cpp/CMakeLists.txt:40。Eigen3/abseil等在上游特定构建分支出现，不应把依赖文件列出的所有测试/示例依赖都当成APK组件。

M8固定WebRTC AAR class major55/minSdk21/arm64 JNI 16KB对齐；实际AGP7.2.2内D8 3.2.74成功转换AAR classes、Kotlin stdlib1.9.25和annotation-jvm1.9.1，无警告。只检查依赖DEX、未构建应用或APK，结果A/verification/m8-video-dependencies.json:1；整个APK仍有ORT的4KB限制。

文档导出工具reportlab/PyMuPDF/Pillow只用于生成PDF和检查排版，不进入应用。Android本机SDK路径、构建缓存与模型副本不纳入Git。

08 CAD、JSON、MJCF与Android模型收集

“JSON转XML”发生在模型制作环节，Android构建消费其产物。



源配置与打包资源是两个阶段；Android的manifest.json也不等于CAD导出配置。

CAD提供装配、零件几何和质量/惯性信息；config_mjcf_groundcontact.json指定Onshape来源、选择/忽略项、关节与后处理规则；onshape-to-robot根据这些信息生成机器人MJCF和网格。这不是通用JSON语法转XML语法，而是把机器人结构和导出规则落实为物理模型。

当前Android入口是scene.xml，它include robot_groundcontact.xml并定义地面及SIT/STAND/INIT关键帧。robot XML头部注明由onshape-to-robot生成。模型来源：T/src/mjlab_microduck/robot/microduck/config_mjcf_groundcontact.json:2、T/src/mjlab_microduck/robot/microduck/robot_groundcontact.xml:2、T/src/mjlab_microduck/robot/microduck/scene.xml:2。

prepare_assets.py递归XML include，读取compiler.meshdir/texturedir，收集mesh/texture/hfield引用文件；保留相对目录，复制到assets/model，并记录每个文件SHA-256。它没有调用CAD服务、JSON导出器、训练器或MJCF改写工具。

当前包是默认groundcontact场景、38个STL网格。walk的scene_walk.xml可供训练阅读或制作模型图，但Android脚本入口固定scene.xml；把“正在跑walk策略”与“必然使用walk碰撞模型”划等号会误解执行效果。资源条目见A/UPSTREAM.lock.json:1。

09 资源收集与手机资源校验的关键代码

文件路径、依赖递归、摘要和运行期落盘各做一件事。

源码：A/scripts/prepare_assets.py:9

```
9 def visit(relative):
 10 relative=Path(relative)
 11 path=(SOURCE/relative).resolve()
 12 if not path.is_relative_to(SOURCE.resolve()): raise ValueError(relative)
 13 if relative in needed: return
 14 needed.add(relative)
 15 tree=ET.parse(path)
 16 compiler=tree.find('compiler')
 17 meshdir=compiler.get('meshdir','') if compiler is not None else ''
 18 texturedir=compiler.get('texturedir','') if compiler is not None else ''
 19 for node in tree.iter():
 20 if node.tag=='include': visit(relative.parent/node.attrib['file'])
 21 elif node.tag in ('mesh','texture','hfield') and 'file' in node.attrib:
 22 folder=meshdir if node.tag=='mesh' else texturedir
 23 needed.add(relative.parent/folder/node.attrib['file'])
 24 visit('scene.xml')
 25 records=[]
```

注解①：解析include时相对当前XML路径继续递归；路径必须留在SOURCE目录内。needed去重，避免重复收集。模型不同层的相对引用要保持，否则mj_loadXML会找不到网格或附加XML。

源码：A/scripts/prepare_assets.py:36

```
36 policy_dest=PROJECT/'app/src/main/assets/policies'
 37 policy_dest.mkdir(parents=True,exist_ok=True)
 38 policies=[]
 39 for name in ('alpha_stand','alpha_walking','roulade'):
 40 source=PROJECT.parent/'Pollen/microduck/policies'/(name+'.onnx')
 41 shutil.copyfile(source,policy_dest/source.name)
 42 policies.append({'name':name,'file':source.name,'sha256':hashlib.sha256(source.read_bytes()).hexdigest()})
 43 (policy_dest/'manifest.json').write_text(json.dumps({'policies':policies},indent=2)+'\n')
```

注解②：策略是直接copyfile，脚本不生成ONNX、没有调用Rust，也没有从Hub下载。三个文件名固定；fresh clone缺少本地未跟踪策略时会在这里失败。

ModelAssets.prepare把打包的model资源安装到应用私有files/model；已有文件摘要正确则跳过，否则用临时文件写入、验摘要再rename。MuJoCo通过这个真实文件路径解析include和网格。关键入口 A/app/src/main/java/com/microduck/sim/ModelAssets.java:16、A/app/src/main/java/com/microduck/sim/ModelAssets.java:41。

三种JSON区别：CAD config决定如何制作模型；model/manifest决定打包文件清单与摘要；policies/manifest和smoke决定策略加载校验。当前它们都不是Android实时控制协议。

10 ONNX的来源、导出与策略校验

训练产物能跨平台使用，训练框架不必随之搬到手机。

正常生产链是microduck_rl训练checkpoint → export.py导出ONNX → 分发/保存为本地文件 → Android收集到assets。当前三份模型来自既有Pollen/microduck/policies本地未跟踪文件；路径和SHA已核对，精确训练run/checkpoint来源未补齐。不要将其写成cargo build生成、或本轮重新训练生成。

源码：T/src/mjlab_microduck/export.py:264

```
264 runner.export_policy_to_onnx(path, filename)
 265
 266 metadata = get_base_metadata(runner.env.unwrapped, run_path=cfg.checkpoint_file)
 267 attach_metadata_to_onnx(onnx_path, metadata)
```

导出器使用runner提供的actor(normalizer(obs))导出路径，并加入metadata；这是部署时保持训练归一化的关键。Android输入的是原始61D观测，不另写一套训练归一化。导出说明 T/src/mjlab_microduck/export.py:1。

文件	当前用途	摘要前缀
alpha_stand.onnx	站立平衡	1569268713e40dee…
alpha_walking.onnx	速度命令步行	e36332d383997d51…
roulade.onnx	单次向前翻滚窗口	3d60da08fc13f29c…

完整摘要见 A/UPSTREAM.lock.json:1。PolicyRunner检查manifest摘要、obs/actions名字、float32及固定[1,61]→[1,14]shape；三策略每个有两组主机smoke输入，启动最大绝对误差必须≤1e-3。入口A/app/src/main/java/com/microduck/sim/PolicyRunner.java:59、A/app/src/main/java/com/microduck/sim/PolicyRunner.java:55。

prepare_assets会为新文件重算manifest；真正防止替换模型却沿用旧对照的另一道检查是smoke里的模型摘要。当前Android只支持feedforward，Rust支持的LSTM/更多skill manifest语义没有全部移植。

11 APK组成与应用启动顺序

资产复制完成后，手机已经不依赖电脑上的Pollen目录。

APK内容	来源/生成方式	运行用途
classes.dex	Java源码及AAR依赖	UI、调度、推理、HTTP与报告逻辑
lib/arm64-v8a/libmujoco.so	MuJoCo源码经NDK编译	MJCF解析、刚体/关节/接触求解
lib/arm64-v8a/libmicroduck_m1.so	C++ JNI/BAM与Rust静态库经NDK链接	Rust Controller/Safety、身体接口、物理推进与视觉快照
ORT AAR带入的native库	Maven依赖合并	CPU网络算子与推理；实际名称/内容由AAR决定
assets/model/*	microduck_rl模型副本	几何、碰撞、惯性、关键帧、相机
assets/policies/*	三份ONNX + manifest/smoke	网络权重、摘要和数值对照
assets/controller.html	Android仓库自带网页	由手机HTTP服务提供控制页面

源码：A/app/src/main/java/com/microduck/sim/NativeSim.java:5

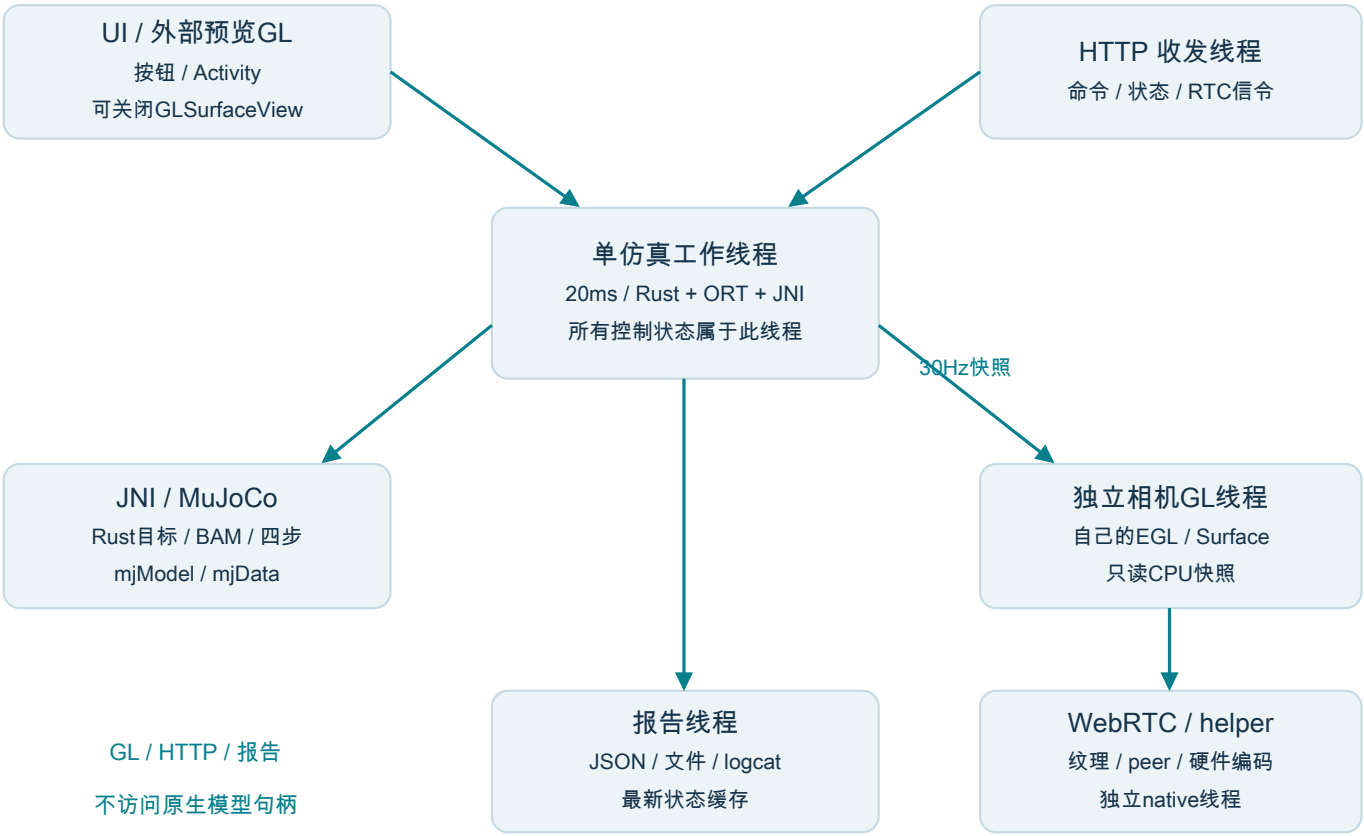
```
5 static {
 6 System.loadLibrary("mujoco");
 7 System.loadLibrary("microduck_m1");
 8 }
 9 static native long open(String xmlPath, String keyframe);
 10 static native String state(long handle);
```

启动时工作线程先ModelAssets.prepare，再NativeSim.open(scene.xml,keyframe)，导出视觉网格，再创建PolicyRunner加载/验证三策略并创建Rust状态。默认BAM、SIT且暂停。策略加载失败时保留基础物理功能并显示错误。见 A/app/src/main/java/com/microduck/sim/MainActivity.java:171。

手机所需的是APK内资源和Android共享库；不会远程调用电脑的Python .venv、Rust robotd，也不会从CAD服务器实时取模型。启动后的files/model及last-report.json属于应用私有目录。

12 运行时技术架构与线程所有权

20ms控制循环保持串行；渲染、网络和报告消耗在各自线程。



MuJoCo、Rust句柄、ORT session和同步回调均属于单仿真工作线程。

线程/模块	拥有的状态	传给其他模块的数据
UI / MainActivity	按钮、视图、Activity生命周期	任务投递；不直接读mjData
单worker	MuJoCo/Rust句柄、ORT Session、raw action反馈、速度与技能状态	不可变场景快照、已捕获报告字段
预览GLES线程	独立GPU网格、外部观察视角	只消费CPU快照；不访问mjData
相机GL线程	独立EGL/Surface、GPU网格、相机FBO	Surface图像；JPEG模式才RGBA读回
WebRTC / helper	peer/track/source、OES纹理/引用计数、硬件编码器	ICE信令与DTLS-SRTP视频；不访问MuJoCo
报告线程	待写最新状态、已完成测试队列	文件/logcat/HTTP JSON缓存
HTTP线程	socket、请求验证、有限队列	命令投递；只读JSON及最新JPEG
JPEG线程	单个采集任务的像素缓冲、Bitmap	最新不可变JPEG

单worker定义和原生所有权说明：A/app/src/main/java/com/microduck/sim/MainActivity.java:39。报告普通状态只保留最新一份待处理，已完成测试按序保存；消费者空闲每50ms检查，不在控制线程每次提交时执行native唤醒。见A/app/src/main/java/com/microduck/sim/ReportDispatcher.java:13。

13 控制周期与JNI物理接口

控制50Hz，物理200Hz，场景快照目标30Hz。

源码：A/app/src/main/java/com/microduck/sim/MainActivity.java:284

```
284 private void controlStep(boolean withVisual) {
 285 long begin=System.nanoTime();
 286 try { if(policyEnabled) policies.tick(handle); }
 287 catch(Exception e) { throw new IllegalStateException("Policy inference failed",e); }
 288 long physicsBegin=System.nanoTime(); policyTickMs=(physicsBegin-begin)/1e6;
 289 NativeSim.advance(handle,4); physicsMs=(System.nanoTime()-physicsBegin)/1e6;
 290 if(policyEnabled) policies.afterStep();
 291 controlMs=(System.nanoTime()-begin)/1e6;
 292 snapshotMs=0;
 293 if(withVisual && (renderEnabled || lan!=null) && visualCadence.due(System.nanoTime())) pushVisual();
 294 controlWithVisualMs=(System.nanoTime()-begin)/1e6;
 295 }
 296 private void pushVisual() {
```

每次控制先按当前状态推理，写入关节目标，再NativeSim.advance(handle,4)。MuJoCo dt=5ms，四步推进20ms。Rust在控制步内推进技能时间；界面完成计数在物理推进成功后更新。control_ms包含策略与物理，control_with_visual_ms再包括本周期可能发生的场景快照。

ticker使用scheduleAtFixedRate(...,20ms)，记录期望开始时间与迟到计数；Java/Android调度不是硬实时保证，50Hz是目标。长时间过载不能据配置排除，已有短期样本和基准需分别看待。调度入口A/app/src/main/java/com/microduck/sim/MainActivity.java:205。

原生open解析MJCF并强制dt=.005；按名字找到14个hinge、其qpos/qvel地址及唯一joint actuator，不假定浮动根或关节索引恒定。root是freejoint，qpos=21/nv=20/nu=14；关键维度见已有手机报告。

代码入口：A/app/src/main/cpp/native_sim.cpp:203、A/app/src/main/cpp/native_sim.cpp:206、A/app/src/main/cpp/native_sim.cpp:219。

一步失败/非有限值/BADQPOS或BADQVEL或BADQACC会抛错并暂停；仿真数值检查与已复用的Rust Safety共同约束目标；它们不能代替真实机器人机械与电机保护。

14 61D观测与14D动作：跨仓库接口契约

维度、顺序、单位和反馈语义都要一致。

索引（左闭右开）	维度	含义
0..3	3	自由根躯干局部角速度
3..6	3	R□×(0,0,-1)重力投影
6..20	14	当前关节角减HOME，rad
20..34	14	同序关节角速度，rad/s
34..48	14	上一帧未缩放/未滤波raw action
48..51	3	vx/vy/yaw速度命令
51..55	4	neck/head pitch、head yaw/roll姿态命令槽
55..61	6	body x/y/z/roll/pitch/yaw；未绑定x/y/yaw强制零

源码：R/duck-control/src/obs.rs:174

```
174 pub fn build(
 175 imu: &ImuData,
 176 joint_positions: &[f64; NUM_JOINTS],
 177 joint_velocities: &[f64; NUM_JOINTS],
 178 home_pose: &[f64; NUM_JOINTS],
 179 last_action: &[f32; ACTION_LEN],
 180 command: &Command,
 181 ) -> Self {
 182 let mut data = [0.0f32; OBS_LEN];
 183
 184 // Carve the buffer into the blocks of the layout table above, by name — as
 185 // *arrays*, so each block's width is in its type and [`fill`] can only be handed a
 186 // source of matching length. `expect` cannot fire: the widths are constants that
 187 // sum to `OBS_LEN`, which `the_layout_widths_sum_to_the_declared_input` pins.
 188 const LAYOUT: &str = "block widths must sum to OBS_LEN";
```

当前Android用户界面只提供速度/单次翻滚；头部与机身命令保留13D布局但不对外提供完整姿态控制。14D输出是关节位置偏移，经过HOME+scale×action得到目标角；不是14个电机转速或14个力矩。

关节顺序：左腿5（hip yaw/roll/pitch、knee、ankle）→头颈4（neck pitch、head pitch/yaw/roll）→右腿5。Android模型无Rust接口的mouth槽，按名字映射主动关节。Android定义 A/app/src/main/cpp/native_sim.cpp:19；Rust布局证据 R/duck-control/src/obs.rs:12。

15 推理、动作滤波与两种执行器

Rust决定目标，Android ORT推理，BAM默认、XML用于对照。

源码：A/app/src/main/java/com/microduck/sim/PolicyRunner.java:97

```
97 void tick(long handle) throws Exception {
 98 double[] result=RustControl.tick(rustCore,NativeSim.bodySample(handle),
 99 new double[]{command[0],command[1],command[2]},filter,this);
 100 NativeSim.bodyGain(handle,(int)result[15]);
 101 NativeSim.bodyTargets(handle,Arrays.copyOf(result,15));
 102 fallen=result[17]!=0;
 103 }
```

选择规则由上游Rust Controller执行：翻滚优先，速度模长≤0.05站立，其余步行。Android ORT继续CPU、单intra/inter-op线程。raw action反馈、缩放和EMA均由Rust执行；NativeSim直接写安全处理后的15槽目标，忽略mouth，无二次滤波。

源码：A/app/src/main/java/com/microduck/sim/PolicyRunner.java:90

```
90 float[] rustInfer(int net,float[] observation) throws Exception {
 91 String[] names={"alpha_stand","alpha_walking","roulade"};
 92 if(net<0 || net>=names.length) throw new IllegalStateException("Unsupported Rust policy");
 93 active=names[net];
 94 long start=System.nanoTime();float[] action=infer(sessions.get(active),observation);
 95 inferenceMs=(System.nanoTime()-start)/1e6;return action;
 96 }
```

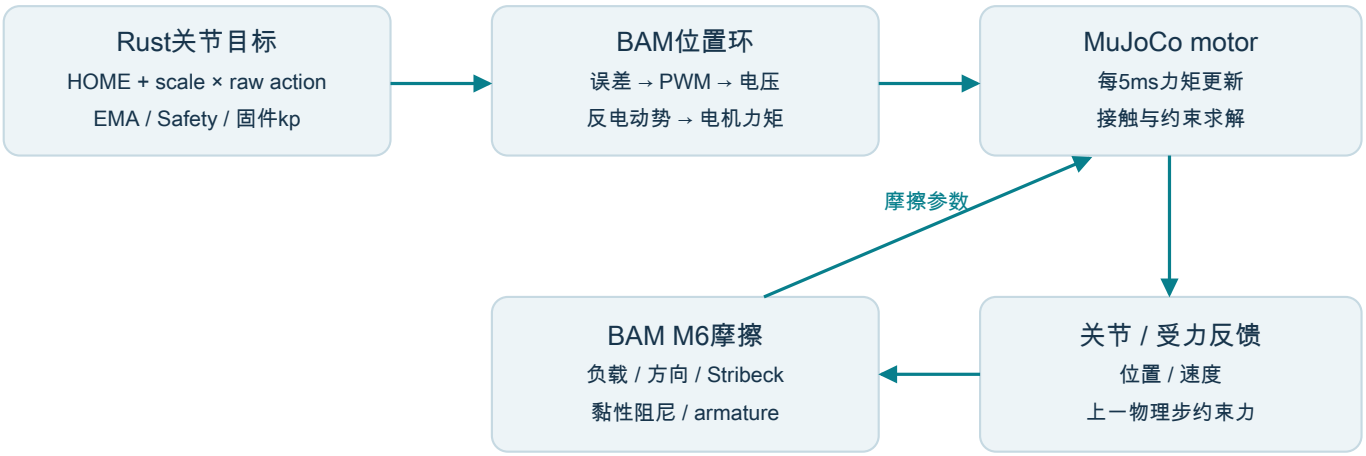
策略	动作scale	XML kp倍率	默认EMA
stand	1.0	0.8	头部α=.5 / 腿α=.7
walk	0.9	1.0	头部α=.5 / 腿α=.7
roulade	1.0	1.0	头部α=.5 / 腿α=.7

EMA为α×新目标+(1−α)×上一目标；第一帧不滤波，切换过滤选项保留状态。Rust给出站立kp160、步行/翻滚kp200；BAM使用固件位置环kp，XML按kp/200缩放原XML gain。XML base kp=.55、站立约.44，不能把这些值与固件kp当同单位比较。

位置伺服源定义 T/src/mjlab_microduck/robot/microduck/robot_groundcontact.xml:41；Rust控制对照 R/robotd/src/control.rs:606。

16 BAM执行器：移植方案与关键公式

M9已经移植，M10继续默认使用；XML保留作对照。



7.4V名义参数；动态电压下垂；训练DR与延迟尚未移植

XML伺服可切回对照；切换暂停并复位SIT

20ms关节目标经过BAM每5ms计算力矩，再由MuJoCo求解运动。

源码：A/app/src/main/cpp/bam_actuator.h:18

```
18 inline double torque(double target,double q,double velocity,double kp,double volts) {
 19 const double duty=std::clamp((target-q)*kp*ERROR_GAIN,-1.0,1.0);
 20 return KT*(volts*duty)/R-KT*KT*velocity/R;
 21 }
```

位置误差与固件kp得到PWM占空比，限制在±1；电压提供驱动力矩，速度通过反电动势减小输出。力矩并不等于raw action，raw action先由Rust转成关节位置目标。

源码：A/app/src/main/cpp/bam_actuator.h:100

```
100 void update(mjModel* m,mjData* d) {
 101 if(!enabled) return;
 102 double load=0; for(double t:previous_torque) load+=std::abs(t);
 103 effective_volts=std::max(VIN_MIN,VIN-DROP_GAIN*load);
 104 for(int i=0;i<14;++i) {
 105 const auto& o=original[i]; int v=o.dof;
 106 double t=powered?torque(targets[i],d->qpos[o.qpos],d->qvel[v],kp,effective_volts):0;
 107 // CPU BAM uses the previous solved forces for the next friction budget.
 108 double external=-d->qfrc_bias[v]+d->qfrc_constraint[v];
 109 for(int e=0;e<d->nefc;++e)
 110 if(d->efc_type[e]==mjCNSTR_FRICTION_DOF && d->efc_id[e]==o.joint) external-=d->efc_force[e];
 111 m->dof_frictionloss[v]=friction(d->qfrc_actuator[v],external,d->qvel[v]);
 112 m->dof_damping[v]=VISCOUS;
```

BAM M6包含方向/负载相关摩擦、Stribeck低速摩擦、黏性阻尼、转子等效惯量和电压下垂。configure将原position执行器转换为motor，并保存XML参数供切回恢复；复位清理上一力矩与目标。切换BAM/XML暂停并复位SIT。

参数/来源	当前值或行为
名义电压 / 下垂 / 最低电压	7.4V / 0.1 / 6V，确定性名义配置

参数/来源	当前值或行为
固件kp	站立160；步行/翻滚200
kt / R	0.36601349688984386 / 2.8113923539223227
来源	Rhoban BAM mjlab_frictionloss，提交62bd8ce…；Apache-2.0
数值证据	1000公式对照约1.1e-16；600步Python/C++轨迹最大误差约1.3e-13

参数与逐文件来源摘要 A/BAM.lock.json:1；模型转换 A/app/src/main/cpp/bam_actuator.h:49；数值报告 A/verification/m9-bam-checks.json:1。

17 训练环境与Android物理的剩余差异

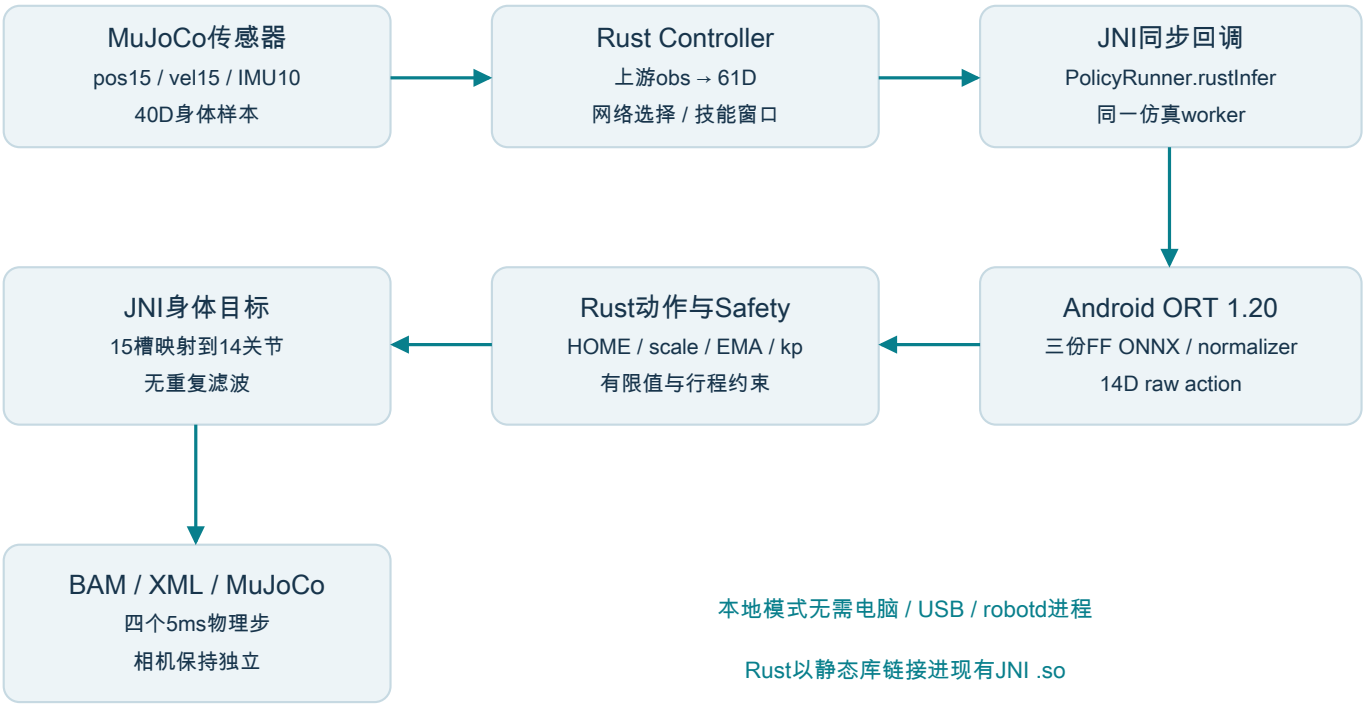
CPU BAM对齐已完成，完整训练环境等价尚未成立。

环节	训练/上游	当前Android M10
网络	actor + normalizer；61D→14D	同ONNX与smoke，Rust组装观测
执行器	BAM XL330 M6	C++移植CPU BAM；默认开启，XML可对照
随机化/延迟	电压/摩擦/惯量等DR，3–6步延迟	名义确定性参数；delay=0，无训练DR
动作滤波	RL约定策略不加low-pass	保留桌面默认EMA头.5/腿.7，可开关
任务完成	奖励和终止衡量训练表现	2秒翻滚窗口，无严格任意姿态成功判定
安全/硬件	完整daemon与真实总线另有功能	Rust Safety目标限制/跌倒报告，无实体舵机驱动

上游约定：T/AGENTS.md:69、T/AGENTS.md:77；Android名义配置 A/BAM.lock.json:32。已完成一圈不能推断完整训练环境等价，也不能推断抬头慢或僵硬已经修复。

18 APK内嵌Rust核心：复用范围

直接编译控制源码，平台相关部分通过适配器接入。



推理后端保留Android ORT；控制决策与状态在Rust

Rust Controller与Safety位于APK内；ORT仍由现有Android后端提供。

源码：A/rust/control-core/src/lib.rs:4

```
4 extern crate self as duck_control;
 5 #[path = "../../../../Pollen/microduck/duck-control/src/model.rs"]
 6 pub mod model;
 7 #[path = "../../../../Pollen/microduck/duck-control/src/imu.rs"]
 8 pub mod imu;
 9 #[path = "../../../../Pollen/microduck/duck-control/src/io.rs"]
 10 pub mod io;
 11 #[path = "../../../../Pollen/microduck/duck-control/src/obs.rs"]
 12 pub mod obs;
 13 #[path = "../../../../Pollen/microduck/duck-control/src/safety.rs"]
 14 pub mod safety;
 15 include!(concat!(env!("OUT_DIR"),"/control_module.rs"));
 16 pub mod policy;
 17 pub use io::Sensors;
```

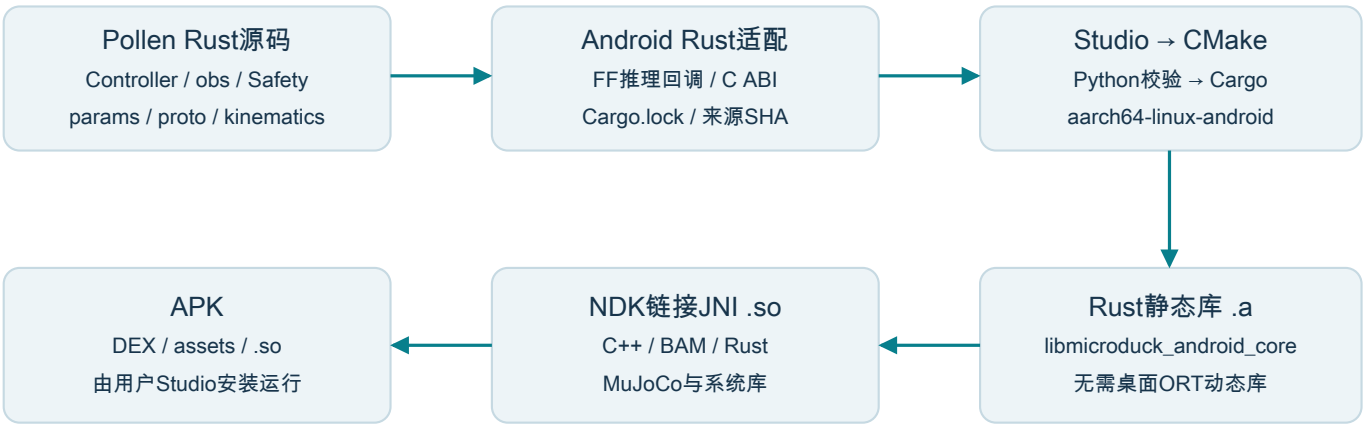
组件	复用方式 / 范围
model/imu/io/obs/safety.rs	通过path直接编译上游源码，保留关节/传感器/61D契约
robotd/src/control.rs	构建目录保留原文，追加取消技能与完成窗口后重启两项Android生命周期钩子
robotd-params/proto/kinematics	Cargo path依赖，复用参数和协议类型

组件	复用方式 / 范围
Android Rust policy.rs	同步ORT回调适配器；三份FF网络；不包含上游桌面加载器
未接入范围	完整robotd进程、全部技能、热加载、命令EMA、温控与硬件服务

取消技能保留raw action与EMA目标；窗口完成后可立即再次翻滚，在途重复请求不延长。代码A/rust/control-core/build.rs:8；固定源文件SHA和许可 A/RUST_CORE.lock.json:1。两个Pollen仓库源码未被Android构建改写。

19 Rust编译与JNI链接的关键代码

Studio构建现在确实调用Cargo，运行期没有桌面daemon。



构建需要Cargo；运行不需要Cargo/Python/电脑

上游文件改变需复核源文件锁与验证，不手工复制桌面二进制

Rust源码编译分支与APK产物；不是将桌面可执行文件直接打包。

源码：A/app/src/main/cpp/CMakeLists.txt:62

```
62 add_custom_target(duck_rust_build
 63 COMMAND "${Python3_EXECUTABLE}" "${M1_PROJECT_ROOT}/scripts/build_rust_core.py"
 64 --cargo "${DUCK_CARGO}" --target-dir "${DUCK_RUST_TARGET_DIR}"
 65 --linker "${ANDROID_TOOLCHAIN_ROOT}/bin/aarch64-linux-android28-clang"
 66 BYPRODUCTS "${DUCK_RUST_ARCHIVE}"
 67 VERBATIM)
 68 add_library(duck_rust_core STATIC IMPORTED)
 69 set_target_properties(duck_rust_core PROPERTIES IMPORTED_LOCATION "${DUCK_RUST_ARCHIVE}")
 70 add_dependencies(duck_rust_core duck_rust_build)
 71 add_library(microduck_m1 SHARED native_sim.cpp rust_control_jni.cpp)
```

CMake先找到Python和Cargo；build_rust_core.py核对上游逐文件SHA，再cargo build --locked --release --target aarch64-linux-android。静态产物libmicroduck_android_core.a链接进libmicroduck_m1.so，不作为第二个Rust共享库打包。

源码：A/app/src/main/cpp/CMakeLists.txt:71

```
71 add_library(microduck_m1 SHARED native_sim.cpp rust_control_jni.cpp)
 72 target_compile_features(microduck_m1 PRIVATE cxx_std_20)
 73 target_compile_options(microduck_m1 PRIVATE -Wall -Wextra -Werror)
 74 target_link_options(microduck_m1 PRIVATE -Wl,-z,max-page-size=16384)
 75 target_link_options(mujoco PRIVATE -Wl,-z,max-page-size=16384)
 76 target_link_libraries(microduck_m1 PRIVATE mujoco duck_rust_core log dl m)
```

NDK链接BAM/JNI、Rust静态库、MuJoCo和Android log/dl/m。实际ARM64共享库链接成功，LOAD段16KB对齐；旧ORT AAR整体页大小兼容限制仍独立存在。

源码校验 A/scripts/build_rust_core.py:10；Cargo入口 A/rust/control-core/Cargo.toml:6；实际原生产物报告 A/verification/m10-android-rust-checks.json:1。源文件锁变化后必须复核并重验，不应直接更新摘要掩盖不兼容变化。

20 Rust与Android ORT：为什么保留混合链路

控制逻辑归Rust，推理后端通过Android接口提供。

当前调用链：Rust选网络并组装61D → JNI同步调用PolicyRunner.rustInfer → Android ORT返回14D动作 → Rust做HOME/scale/EMA/Safety → JNI身体目标。所有调用在同一仿真worker，不保留跨周期Java对象指针。

源码：A/rust/control-core/src/policy.rs:18

```
18 pub fn infer(&mut self,obs:&Observation,net:Net)->Result<[f32;ACTION_LEN],PolicyError>{
 19 let index=match net{Net::Stand=>0,Net::Walk=>1,Net::Skill(0)=>2,_=>return Err(PolicyError)};
 20 let host=self.0.borrow().ok_or(PolicyError)?;
 21 let mut action=[f32::NAN;ACTION_LEN];
 22 let result=unsafe{(host.callback)(host.context,index,obs.as_slice().as_ptr(),action.as_mut_ptr())};
 23 if result!=0 || action.iter().any(|v|!v.is_finite()){return Err(PolicyError)}
 24 Ok(action)
 25 }
 26 }
```

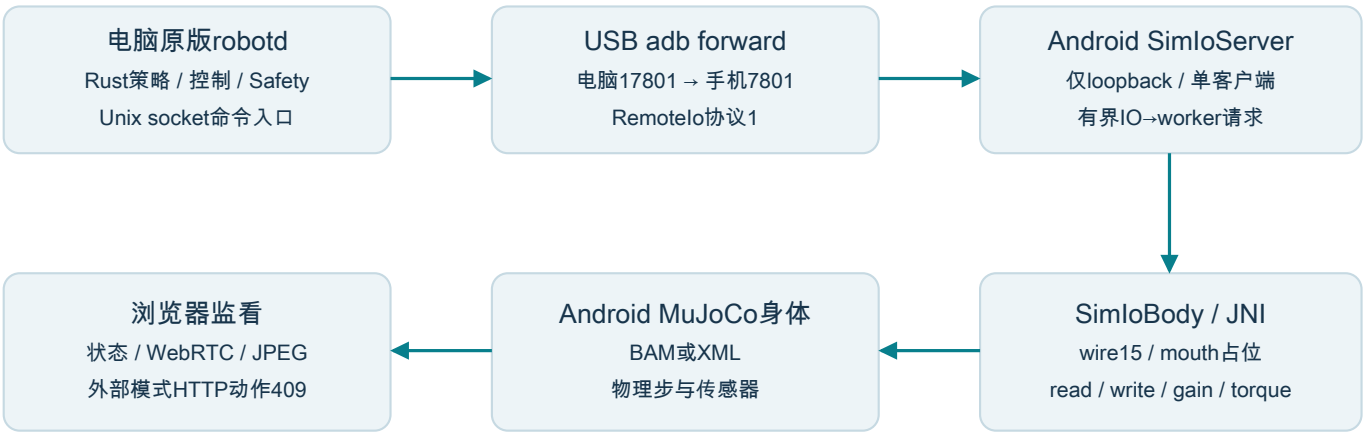
JNI校验sensor40/twist3与action14，Java推理异常保留并返回worker；Rust拒绝非有限传感器/输出，失败不提交身体目标。句柄只在创建线程有效，关闭或跨线程调用拒绝。依据 A/app/src/main/cpp/rust_control_jni.cpp:10、A/rust/control-core/src/lib.rs:62。

Rust直接调用ORT通常也是绑定底层ORT引擎，并非纯Rust推理重写。它可以统一会话/热加载/循环状态管理并减少一次Java回调，但当前没有性能证据证明这一跳是瓶颈。已实机通过的Android ORT继续保留；后续按模型管理或测量结果决定是否迁移。

Safety跌倒状态只报告，不自动中断翻滚；本地intent_age传0，浏览器断线由Android租期处理。复用Safety并不等于完整daemon跌倒恢复或另加Rust通信死手保护。源码 A/rust/control-core/src/lib.rs:105。

21 SimIo身体服务：桌面原版控制手机

外部模式保留，用于源码对照与完整daemon联调。



手机本地按钮接管：关闭外部服务，再运行内嵌Rust

断线暂停并关闭力矩；两种控制器互斥

相机仍走Android视频链，无mediad / GStreamer依赖

电脑Rust→USB转发→Android身体；与手机内嵌控制互斥。

项目	实现
监听 / 传输	手机127.0.0.1:7801；USB adb forward电脑17801→7801
协议	换行JSON；hello protocol1 joints15；read/write/gain/torque/slow
关节映射	mouth为wire9占位；右腿wire10→model9；14D模型↔15槽协议
线程/请求	独立IO线程；JNI只在仿真worker；150ms有界等待，超时取消排队操作
控制权	外部模式停本地策略；HTTP动作409、网页只监看；手机按钮接管关闭服务
断线 / 传感器	断线关力矩并暂停；gyro/gravity/quat和关节实测；无假电流负载

slow里的7.4V/32℃为模拟占位；torque:false在首次释放后表示受物理作用的limp，TCP断线则暂停。没有接入ToF扩展或mediad相机协议，视频仍由Android相机服务提供。

代码 A/app/src/main/java/com/microduck/sim/SimIoServer.java:50、A/app/src/main/java/com/microduck/sim/SimIoBody.java:18、A/app/src/main/cpp/native_sim.cpp:116。

22 两种Rust模式的使用命令

日常本地运行；需要桌面对照时才开启身体服务。

模式	手机	电脑
内嵌Rust（默认）	M10直接点站立/前进/翻滚；可开启LAN和相机	无需运行脚本或连接USB
外部Rust SimIo	保持前台，开启Rust身体服务	adb转发，再运行robotd帮助脚本并enable

执行位置：工作区；手机先手动开启Rust身体服务

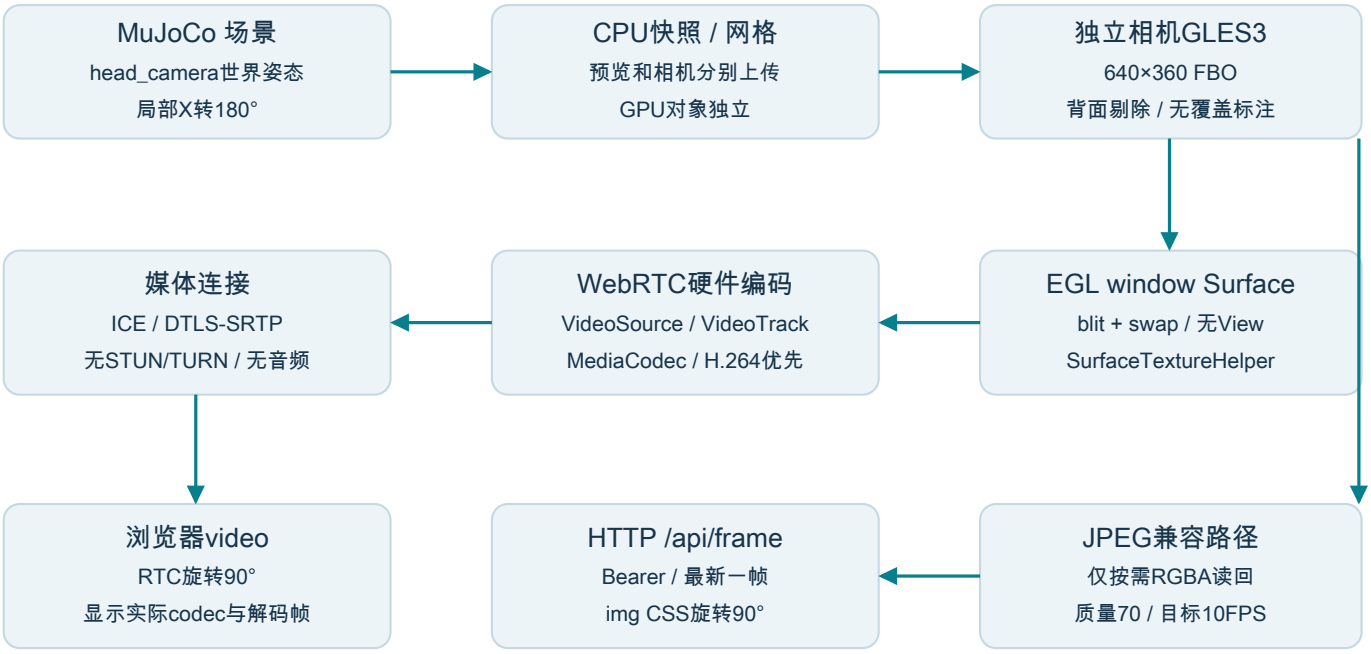
```
cd android-sim
adb -s <SERIAL> forward tcp:17801 tcp:7801
python3 scripts/android_robotd.py run
# 另一个终端，同样位于android-sim
python3 scripts/android_robotd.py enable
python3 scripts/android_robotd.py walk --seconds 3
python3 scripts/android_robotd.py disable
```

walk默认vx=0.30m/s，与手机/网页一致。0.15几乎不走的现象已在早期主机闭环和M9用户反馈出现；脚本只改默认值，显式低速仍原样发送。0.30是命令值，不等于测得速度。证据 A/scripts/android_robotd.py:29、A/verification/user-M9-simio-feedback.json:1。

脚本复用现有桌面robotd；电脑加载完整七策略，坐姿启用需要sitstand。三份Android模型之外的策略不加入APK。结束Ctrl-C退出daemon，手机断线暂停；可adb forward --remove tcp:17801。详情 A/BAM_SIMIO.md:1。

23 CAD渲染与机器人相机架构

物理模型与视觉数据相同来源，绘制实现由Android完成。



HTTP /api/rtc 交换信令；HTTP命令与3秒控制心跳保持

关闭手机3D可继续相机；M8用户反馈实机验证正常

当前视频是head_camera的独立离屏渲染，不是观察窗口投屏。

Native层从MuJoCo编译后的网格构建索引顶点/法线，保持硬边；场景快照携带每个可见geom的mesh id、世界位置/旋转、颜色与接触点。每个快照没有mjData指针；Java/GLES依据数组绘制。

快照v2结构：9float头 + 每geom17float + 每contact3float + 相机尾16float。相机尾包括存在/FOV/位置3/旋转9/near/far。MuJoCo row-major 3×3转GLES column-major 4×4由SceneFrame处理。依据A/app/src/main/cpp/visual_snapshot.h:14、A/app/src/main/java/com/microduck/sim/SceneFrame.java:26。

相机姿态在快照中右乘diag(1,-1,-1)，沿用桌面local X 180°修正，不修改MJCF。GLES用独立640×360 FBO、模型FOV/裁剪重新绘制；JPEG质量70、目标10FPS；网页再顺时针90°。

相机灰屏修复只在相机pass启用GL_BACK/GL_CCW背面剔除，避免镜头/头壳内部背面遮挡。手机观察视角保持独立、可拖动。M8由预览与相机分别实例化SceneRenderer，各自拥有GPU网格与EGL上下文；关闭手机3D仍更新相机快照。相机的独立Surface由SurfaceTextureHelper承接，硬件编码器通过共享纹理上下文消费；离开应用仍暂停并关闭LAN。

关键来源：A/app/src/main/java/com/microduck/sim/SceneRenderer.java:148；A/app/src/main/java/com/microduck/sim/VideoCapture.java:72；桌面对照 T/src/mjlab_microduck/sim/camera.py:94、R/mediad/src/main.rs:229。

WebRTC目标20FPS，HardwareVideoEncoderFactory优先H.264、上限1.5Mbps；JPEG保留目标10FPS/质量70作手动兼容模式。RTC纹理路径不执行应用层glReadPixels/JPEG。SurfaceTextureHelper处理纹理引用计数和BufferQueue/EGL同步，占用时跳过采集；是否实际Surface编码仍需设备日志。RTC携带90°旋转，浏览器video不

再叠加CSS旋转。

源码：A/app/src/main/java/com/microduck/sim/VideoCapture.java:49

```
49 textureHelper=SurfaceTextureHelper.create("microduck-rtc-texture",egl.getEglBaseContext());
 50 if(textureHelper==null) throw new IllegalStateException("Texture helper creation failed");
 51 textureHelper.setTextureSize(HeadCamera.WIDTH,HeadCamera.HEIGHT);
 52 textureHelper.setFrameRotation(HeadCamera.MOUNT_ROTATE);
 53 // The camera has its own window Surface, backed by a SurfaceTexture, with no UI owner.
 54 egl.releaseSurface(); egl.createSurface(textureHelper.getSurfaceTexture()); egl.makeCurrent();
 55 if(!closed) {
 56 rtc=new RtcVideo(context,egl.getEglBaseContext());
 57 textureHelper.startListening(frame->{ RtcVideo video=rtc; if(video!=null) video.frame(frame); });
 58 }
 59 glReady=true;
```

独立相机初始化入口 A/app/src/main/java/com/microduck/sim/VideoCapture.java:34；FBO到Surface A/app/src/main/java/com/microduck/sim/SceneRenderer.java:183。两项均为M8源码实现，用户已反馈实机验证正常；反馈未附实际codec或硬件编码器日志。

24 浏览器控制、状态与视频传输

控制用HTTP；WebRTC媒体用ICE/DTLS-SRTP，JPEG作为兼容模式。

接口	行为	访问码
GET /	手机提供controller.html	无需；连接时填写
GET /health	服务/协议检查	无需
GET /api/state	读取报告线程维护的JSON缓存	Bearer
GET /api/frame	最新JPEG；不可用时503	Bearer
POST /api/command	校验并投递command，执行后确认	Bearer
POST /api/rtc	open/poll/close；offer/answer、增量ICE与会话租期	Bearer

JSON命令	作用
{"op":"stand"}	启用站立并运行
{"op":"walk","vx":0.30,"vy":0,"yaw":0}	设置速度并运行
{"op":"roulade"}	一次2秒翻滚窗口
{"op":"pause"} / {"op":"reset","pose":"SIT"}	暂停 / 关键帧复位并关闭策略
{"op":"heartbeat"}	续租已有远程运行；不会自行恢复/启动

Socket使用2个处理线程、4个等待连接；HTTP头上限4096byte、命令体512byte、RTC信令体32KiB，读阶段2秒、处理起总期限3秒。RemoteMailbox最多一条等待确认命令，1秒超时未执行时可取消；已开始操作不能随断开回滚。HTTP线程不执行JNI/ORT。

浏览器远程运行需3秒租期内心跳；本地按钮释放远程控制。Activity离开前台暂停并关闭LAN，回前台需手动开启新会话。服务访问码每次用SecureRandom生成4位数字（0000–9999），网页保留前导零；不进入普通状态或验证记录。内部RTC观看者id仍为32位十六进制。JPEG只保留一个在途任务和最新帧，2秒无请求停采集；RTC只允许一个观看者、8秒无poll释放peer。每个回调匹配generation，旧close不关新peer，取消id避免迟到open恢复会话。浏览器关闭/隐藏时关peer并尽力发close。

代码：A/app/src/main/java/com/microduck/sim/LanControlServer.java:42、
A/app/src/main/java/com/microduck/sim/RemoteMailbox.java:1、
A/app/src/main/java/com/microduck/sim/RemoteLease.java:5、
A/app/src/main/java/com/microduck/sim/MainActivity.java:571。

RTC信令不经控制串行队列；browser创建recvonly offer并先收集本地ICE，Android异步设置远端/创建answer，poll返回answer及增量候选。没有STUN/TURN或音频，限同一局域网；HTTP连通不证明媒体连通。实际协商与编码器由rtc_codec、rtc_encoder_implementation和接收端framesDecoded验证，不以硬件优先设置代替实测。

入口 A/app/src/main/java/com/microduck/sim/RtcVideo.java:93；会话隔离
A/app/src/main/java/com/microduck/sim/RtcSession.java:9；固定依赖 A/VIDEO_DEPENDENCIES.lock.json:1。

25 单次前滚翻与当前抬头问题

已通过一圈；动作质量诊断尚未等同修复。

源码：A/app/src/main/java/com/microduck/sim/RouladeWindow.java:5

```
5 static final int TICKS=100; // RL deployment rehearsal gives roulade 2 seconds at 50 Hz.
 6 private int remaining;
 7 private long completed;
 8 boolean active() { return remaining>0; }
 9 int remaining() { return remaining; }
 10 long completed() { return completed; }
 11 boolean start() {
 12 if(active()) return false; // Repeated taps cannot keep the roll running indefinitely.
 13 remaining=TICKS; return true;
 14 }
 15 void advance() { if(remaining>0 && --remaining==0) completed++; }
 16 void cancel() { remaining=0; }
```

Rust控制器执行100个20ms周期窗口，即2秒仿真时间；第101周期选择stand。Java RouladeWindow仅计数与请求去重，在物理推进成功后更新。重复点击不延长，暂停冻结，站立/步行取消，reset清空。completed计窗口完成，不是成功翻滚次数。M7曾用旧原型1秒，用户反馈不足一圈；M7.1改为CPU推理入口2秒，策略权重和倍率未变。

M7.1用户已确认整圈翻滚，并纠正问题为“抬头慢，整体动作僵硬”。只读设备日志约1.42秒采样已近360°投影、机身直立、双脚接触，仍需等到2秒交接。推理采样约0.81–1.12ms，控制加快照4.44–9.81ms，日志约250ms一份，不能据此排除瞬时卡顿。

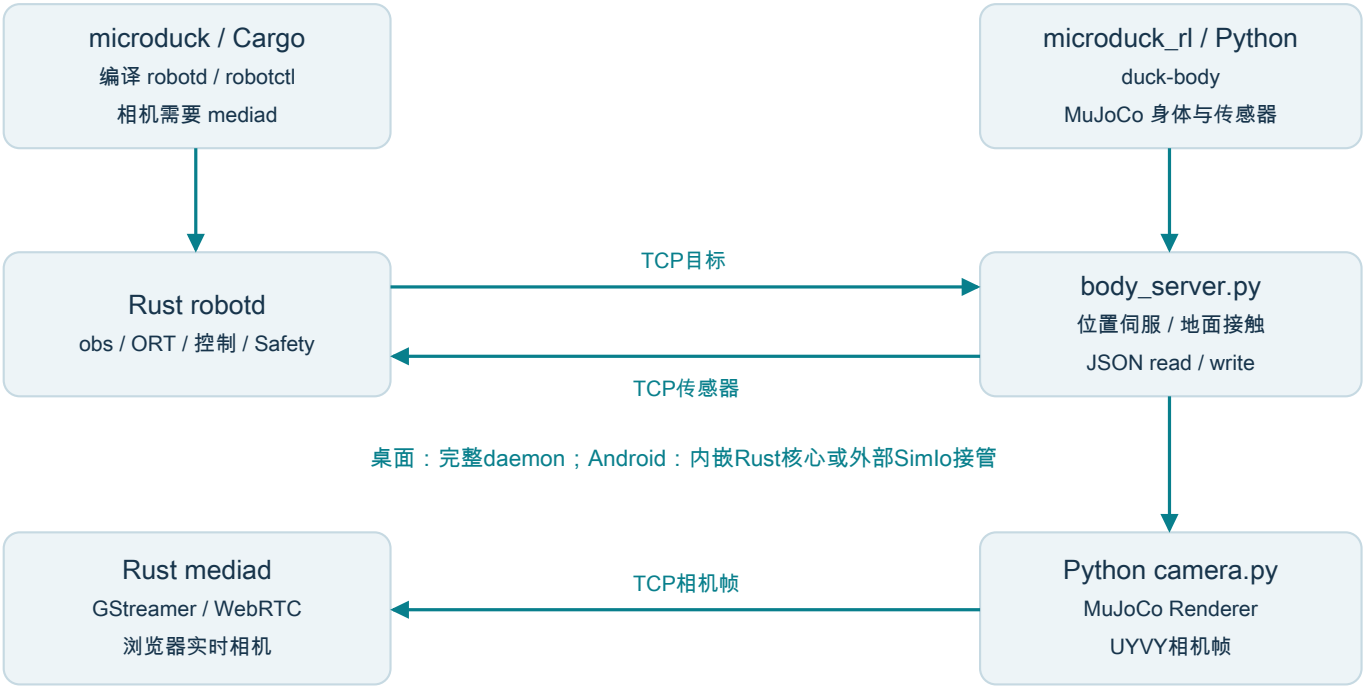
主机同ONNX/XML伺服对照：交接时neck/head_pitch约−1.45/1.35rad；颈部先回，头部仍折叠，相机几何仰角先下沉再恢复，头关节回到自身最终姿态0.1rad内约0.50秒。仅关闭恢复头部滤波约0.46秒且视线下沉仍存在。

下一轮应先对照交接阶段raw/filtered目标、实际角度和力矩，再决定头颈协同或执行器方案。原手机日志缺关节数据，主机不是其精确动作重放；当前没有直接缩短窗口、覆盖头目标或加大增益。

证据：A/verification/device-M71-roll-acceptance.json:1、A/verification/device-M71-roll-log-samples.json:1、A/scripts/head_recovery_rehearsal.py:1、A/verification/host-M71-head-recovery.json:1。

26 与桌面duck-sim的编译/运行关系

两套方案都复用模型与策略，控制执行主体不同。



桌面真实Rust daemon通过TCP连接Python身体；Android内嵌同一Rust控制源码；外部Simlo也可让电脑原版daemon接管。

scripts/duck-sim up会调用cargo build构建robotd等服务，并用microduck_rl/.venv中的Python启动mjlab_microduck.sim.body_server。Rust通过Simlo用TCP JSON read/write/gain等协议访问模拟身体；身体端用CPU MuJoCo推进世界。它不是PPO训练器在实时运行。

有相机时，Python camera.py用MuJoCo Renderer产生帧，经TCP输出给Rust mediad；mediad走GStreamer/WebRTC。macOS曾报缺GStreamer就是这条桌面媒体链的依赖问题；当前Android自写GLES/独立Surface，并以Android WebRTC AAR/MediaCodec与JPEG兼容模式实现视频，不使用mediad或GStreamer。

关键证据：R/scripts/duck-sim:196、R/scripts/duck-sim:637、R/duck-control/src/sim.rs:1、T/src/mjlab_microduck/sim/body_server.py:94。

HTTP /api/command、桌面Unix socket JSON-RPC和Simlo TCP是三种不同接口。M9已接入Simlo身体协议，M10本地使用JNI；Android视频仍不走mediad。

27 训练插件与新策略接入流程

训练环境与应用构建环境分别管理。

microduck_rl通过mjlab.tasks entry point加载mjlab_microduck.tasks并注册任务。mjlab提供向量化训练环境/管理器，MuJoCo Warp提供GPU物理，PPO通过rsl_rl训练actor；模型导出后手机只执行actor推理，奖励、critic和PPO更新不会随ONNX在手机里运行。插件发现证据 T/pyproject.toml:40、T/src/mjlab_microduck/tasks/__init__.py:1。

执行位置：工作区根目录开始；仅命令模板，本轮未执行

```
cd Pollen/microduck_rl
uv sync
uv run list-envs
# 用list-envs确认TASK_ID，训练机需要适配的CUDA环境
uv run train <TASK_ID> --env.scene.num-envs 64 --agent.max_iterations 5
uv run train <TASK_ID> --env.scene.num-envs 4096
uv run play <TASK_ID> --wandb-run-path <entity/project/run_id>
uv run scripts/export.py <TASK_ID> --wandb-run-path <entity/project/run_id>
```

先做64环境/5迭代smoke，再长训，来自上游约定 T/AGENTS.md:26。更换模型需核对61D/14D、关节顺序、command语义、normalizer、支持的feedforward格式；reward修改只有重新训练导出后才影响Android。

接入Android：保存到Pollen/microduck/policies约定文件名 → 运行prepare_assets.py → 在主机用policy_rehearsal.py生成新smoke并复演 → 更新来源/摘要记录 → Studio构建 → 用户验证动作。如果新增第四类动作，还要修改资源收集、PolicyRunner会话集合、Rust policy适配器/技能配置及按钮/远程协议，不能只多放一个ONNX。

28 新机器恢复工程与Studio编译

三份本地策略必须另行准备；两仓库clone不自动获取它们。

执行位置：新工作区；既有工作区有未提交改动时不要直接照抄checkout

```
mkdir microduck-workspace
cd microduck-workspace
git clone git@github.com:luoyanfeidao/microduck_android.git android-sim
git clone https://github.com/pollen-robotics/microduck.git Pollen/microduck
git clone https://github.com/pollen-robotics/microduck_rl.git Pollen/microduck_rl
git -C Pollen/microduck checkout a9ec4b2079ef8ee7904014089c885bb07d57d63c
git -C Pollen/microduck_rl checkout cb70b792312d559a4da09064d92009079671815f
```

把与UPSTREAM.lock一致的alpha_stand.onnx、alpha_walking.onnx、roulade.onnx放入Pollen/microduck/policies。这些文件及自动生成的XML/STL副本没有上传Android Git；单独clone android-sim不足以离线构建。

Studio步骤	配置/结果
打开工程	打开android-sim目录，使用其Gradle Wrapper
Gradle JDK	当前JBR17.0.14（JDK17），Java源码/输出仍release11
SDK Manager	Platform34、NDK25.1.8937393、CMake3.30.3
local.properties	由Studio生成本机sdk.dir；不从别人机器照搬绝对路径
Rust目标	rustup target add aarch64-linux-android；本机已安装，验证工具链1.98.1
同步/构建	Git/Python/Cargo可用；首次下载原生源码、Cargo依赖与AAR
Run	选app和ARM64/API28+设备；本项目当前验证设备是4KB页Android16

复用现成模型和策略构建不要求uv sync，不要求安装CUDA/PyTorch/BAM，但M10需要Cargo与Android Rust目标。主机验收/重生成smoke则另需相应Python环境。来源：A/DEPENDENCIES.md:1、A/README.md:38。

29 运行使用与只读排查命令

先启用策略，再观察；两台设备分别承担仿真与浏览器控制。

操作	预期行为
初始启动	模型/策略校验，SIT暂停；body16/nq21/nv20/actuator14/mesh38/dt.005
SIT/STAND/INIT	关键帧物理姿态复位，暂停并关闭策略；不是同名站立策略
策略站立	首次从HOME启用ONNX并运行，持续平衡
前进0.30m/s	切walking并保留已启用时的姿态/历史
向前翻滚一次	建议先站稳；2秒仿真窗口后切stand继续运行
局域网控制	同Wi-Fi或热点，浏览器打开手机显示地址并配对
开启机器人摄像头	仿真应用保持前台；3D可关闭，浏览器选择RTC/JPEG并勾选相机

“策略测试10s”是不等待墙钟的500周期计算基准，不显示每帧动画；“实时测试30s”是墙钟测试，结束自动暂停。性能/温升/前后台验收已按用户要求延期，当前不以重跑测试作为功能开发前置条件。

执行位置：已由用户Studio运行的debug应用；只读设备状态，不安装或启动应用

```
adb devices -l
adb -s <SERIAL> shell dumpsys package com.microduck.sim
adb -s <SERIAL> shell run-as com.microduck.sim cat files/last-report.json
adb -s <SERIAL> logcat -d -s MicroduckM1:I "*:S"
```

普通last-report可能被后续状态覆盖；已完成测试另存last-policy-benchmark.json/last-physics-benchmark.json/last-realtime-test.json。不要把旧测试当当前动作状态，或把onStop前报告当后台即时证据。写入入口A/app/src/main/java/com/microduck/sim/MainActivity.java:433。

执行位置：可选的电脑辅助链路；手机先手动开启LAN，第二台Android仍使用Wi-Fi地址

```
adb -s <SERIAL> forward tcp:18080 tcp:8080
curl http://127.0.0.1:18080/health
adb -s <SERIAL> forward --remove tcp:18080
```

30 变更如何传到Android：重建关系表

修改上游文件不等于正在运行的APK已经更新。

修改内容	还需做什么	是否要训练 / 重编APK
CAD / 导出JSON	先重新导出robot XML/STL，再收集；核对质量/碰撞与策略是否仍适配	通常要重编APK；显著物理变化需重新评估/可能重训
scene或robot XML/STL	Android下次preBuild复制新版本；检查manifest与物理结果	重编APK；不一定必须重训
任务reward/obs/events	训练得到新checkpoint、按标准路径导出ONNX，再接入	仅改Python不改变APK；通常需重训/重导出/重编
ONNX权重	新manifest与smoke；来源记录及动作闭环重验	已有新ONNX时无需在手机训练；重编APK
已复用Rust模块	审查上游修改，更新RUST_CORE.lock并重验；Studio自动Cargo编译	重编APK；完整daemon未接入模块不自动进入APK
body_server.py/camera.py	核对节拍、相机等变化是否需同步到Android	Python改动本身不被Android打包
Android Java/JNI/网页	源码检查、Studio构建；浏览器刷新并重连新会话	重编APK；通常无需训练
MuJoCo/ORT版本	更新固定来源/补丁/工具链及数值、兼容性验证	重编APK并重新验收

当前已有外部Rust SimIo和内嵌Rust；尚无APK内模型热更新或在线下载。模型路径是build-time资源输入，native模型在启动时加载；已安装APK不跟随电脑文件变化。

三份策略与model副本的Git排除：A/.gitignore:7；固定来源记录 A/UPSTREAM.lock.json:1；编译前复制入口 A/app/build.gradle:40。

31 检查证据与当前验收边界

阶段性功能完成；用户确认与自动化证据分开记录。

证据	支持的结论	边界
user-M10-feedback.json	用户已实机验证M10通过	无新增日志/codec、温升或细分速度测量
m10-checks.json	JDK17/release11、真实AAR、ARM64 JNI、HTTP/RTC/SimIo回归	主机源码与协议检查
m10-android-rust-checks.json	实际NDK/CMake Rust链接，JNI符号与16KB LOAD段	原生产物检查；不代表完整APK页大小验收
m10-embedded-rust-checks.json	生产Java/JNI/Rust/ORT闭环；3秒前进约0.254m，翻滚約359°恢复	HOME起始主机复演；非手机动作/速度测量
m9-bam-checks.json	1000公式与600步Python/C++ BAM对照，XML恢复	确定性CPU BAM，不含训练DR/延迟
user-M9-simio-feedback.json	USB Rust连接通过；0.30明显行走	低速表现仍待解决
历史M7/M8与M54证据	相机、整圈翻滚及旧30秒预算的已有验收	不跨版本推断M10性能与动作质量

完整来源：A/verification/user-M10-feedback.json:1；A/verification/m10-checks.json:1；A/verification/m10-android-rust-checks.json:1；A/verification/m10-embedded-rust-checks.json:1；A/verification/m9-bam-checks.json:1。

尚未验证：低速速度跟踪、抬头/僵硬的关节级归因、任意起始姿态翻滚、完整训练DR/延迟、完整daemon功能、精确ONNX训练run/checkpoint来源、详细codec/编码器日志、16KB页整APK兼容、长期性能/温升/生命周期。性能相关专项继续按用户要求延期。

32 常见构建与运行问题定位

按失败发生的层次排查，减少误改模型与构建链。

现象	优先检查	对应层
prepareModelAssets失败	Studio PATH里的python3、相对Pollen目录、scene/includes/STL存在、三份ONNX存在	资源输入/生成
MuJoCo下载或revision失败	Git/网络、固定标签/提交、本机.native-cache源码是否被改到另一版本	CMake源代码准备
Cargo/源文件锁失败	Rust/Cargo路径、Android目标、RUST_CORE.lock与当前上游文件是否匹配	Rust编译/来源校验
NDK补丁不能应用	MuJoCo源码是否为固定版本、是否存在额外未审计改动	兼容补丁
smoke checksum或shape失败	新模型是否匹配smoke、61D/14D和名字/float类型、是否feedforward	策略格式/来源
模型可见但不能站稳	HOME/关节顺序/反馈语义、动作倍率/过滤、XML vs BAM执行器	控制闭环
浏览器打不开	LAN是否开启、同网段/热点、显示IP、客户端隔离、前台是否已退出	网络连接
HTTP401 / 503	401查本次访问码；503查独立camera_gl_ready/error或RTC初始化	认证 / 画面供给
RTC无图而JPEG正常	查rtc_hardware_codecs/error/connection、同网设备互通与编码器日志	信令/ICE/硬件编码
产帧但画面全灰	真实相机姿态、模型背面遮挡与剔除；不能只据frames/error=0判正常	相机几何/绘制
翻完抬头慢	交接期间头颈raw/filtered目标、实际角度、相机方向与伺服响应	动作质量；采样日志不足以归因

重新生成资源的手动命令为python3 scripts/prepare_assets.py（在android-sim），可单独排查Python/文件输入；正常Studio构建会自动执行。不要把pip安装mjlab作为解决缺ONNX/STL的默认手段，也不要把桌面GStreamer报错引入当前Android JPEG链。

33 阶段性交付与项目价值

M10可以固定为第一阶段交付基线。

已形成Android独立机器人仿真与控制平台：真实物理模型、三种策略、BAM、Rust核心、机器人相机与双设备控制均接通，并获得用户M10实机功能确认。可以作为可演示、可扩展的工程原型；动作质量与长期稳定性仍有后续目标。

价值	已有能力	下一步积累
降低实验门槛	无需实体机器人即可观察策略、关节运动与机器人视角	场景扰动与可重复任务
建立部署路径	训练资源/ONNX→Rust→Android原生物理与视频	模型导入、版本切换与失败回退
验证控制逻辑	同一手机身体接受内嵌Rust或桌面原版控制	记录/回放观测、动作、目标和实际轨迹

M10基线源码 A/app/build.gradle:11；用户验收 A/verification/user-M10-feedback.json:1。仿真能筛查接口和闭环问题，但不能替代实体机器人的机械、电气和实际运动验证。

34 后续方向与优先顺序

先提升动作质量和可诊断性，再按需求扩展。

优先级	方向	可评估的结果
第一	低速、头颈恢复与动作自然程度	记录关节目标/实际/力矩；量化速度、交接视线与跟踪误差
第一	调试与回放	50Hz观测/raw action/目标/关节/策略切换；两种控制模式可对照
第二	策略接入与模型管理	导入校验、版本切换、失败回退；减少改代码打包
第二	场景与任务	坡面、摩擦、障碍和外力；检验适用范围
后续	性能与稳定性	控制预算、视频并发、温升、生命周期；按既定延期安排
按需	扩大Rust复用/直接ORT调用	更多技能/热加载/循环状态，依据实际需求及性能证据

BAM执行器、独立相机GL/Surface、WebRTC以及Rust/SimIo已经实施，属于当前基线，不能继续列为“未做”。建议下一阶段把动作质量与调试回放结合，获得可解释的优化依据。

35 开发方法与接续阅读

沿数据链确认原因，每个结论绑定版本和证据。

先确认实际调用方、观测、raw action、目标与响应，再做单变量对照。维度正确不代表关节语义正确；推理成功不代表动作成功；产帧成功不代表视野正确。主机对照、用户确认和设备日志分别表述，不把旧版本性能搬到新版本。

阅读顺序：app/build.gradle → CMake/Cargo/build_rust_core → PolicyRunner.tick → Rust适配器 → 上游Controller/obs/Safety → NativeSim.bodyTargets → BAM/物理 → visual_snapshot/SceneRenderer → VideoCapture/RTC/HTTP。资源链另读prepare_assets和ModelAssets。

配套入口：A/RUST_CORE.md、A/BAM_SIMIO.md、A/README.md、A/DEPENDENCIES.md、A/NETWORK.md、A/VIDEO.md和工作区HANDOFF.md。本PDF与Markdown统一从docs/content.py生成；每处file:line及逐字代码摘录从真实源码定位，并保存SHA。

36 附录：源码路径与证据索引

行号对应本地快照；代码摘录与源码摘要一起保存。

sources.json保存每处引用的完整路径、定位锚点、行号与SHA-256，以及所有逐字源码摘录。构建脚本自动检查锚点唯一；解释文字仍需在源码更新后人工复核。上游未跟踪ONNX的训练run/checkpoint来源尚未补齐，本文不据目录名推断其训练历史。

缩写	工作区相对根目录
A/	android-sim/
R/	Pollen/microduck/
T/	Pollen/microduck_rl/
M/	android-sim/.native-cache/mujoco-3.10.0/

文件（工作区相对路径）	引用行号
Pollen/microduck/.cargo/config.toml	24
Pollen/microduck/Cargo.toml	31
Pollen/microduck/duck-control/src/obs.rs	12, 174
Pollen/microduck/duck-control/src/sim.rs	1
Pollen/microduck/mediad/src/main.rs	229
Pollen/microduck/robotd/src/control.rs	606
Pollen/microduck/scripts/duck-sim	196, 637
Pollen/microduck_rl/AGENTS.md	26, 69, 77
Pollen/microduck_rl/pyproject.toml	15, 40
Pollen/microduck_rl/src/mjlab_microduck/export.py	1, 264
Pollen/microduck_rl/src/mjlab_microduck/robot/microduck/config_mjcf_groundcontact.json	2
Pollen/microduck_rl/src/mjlab_microduck/robot/microduck/robot_groundcontact.xml	2, 41
Pollen/microduck_rl/src/mjlab_microduck/robot/microduck/scene.xml	2
Pollen/microduck_rl/src/mjlab_microduck/sim/body_server.py	94
Pollen/microduck_rl/src/mjlab_microduck/sim/camera.py	94
Pollen/microduck_rl/src/mjlab_microduck/tasks/__init__.py	1
android-sim/.gitignore	7
android-sim/BAM.lock.json	1, 32
android-sim/BAM_SIMIO.md	1
android-sim/DEPENDENCIES.md	1
android-sim/README.md	38, 45
android-sim/RUST_CORE.lock.json	1, 6
android-sim/UPSTREAM.lock.json	1
android-sim/VIDEO_DEPENDENCIES.lock.json	1
android-sim/app/build.gradle	5, 11, 12, 21, 24, 30, 31, 40
android-sim/app/src/main/AndroidManifest.xml	3

文件（工作区相对路径）	引用行号
android-sim/app/src/main/cpp/CMakeLists.txt	7, 26, 40, 52, 62, 71
android-sim/app/src/main/cpp/bam_actuator.h	18, 49, 100
android-sim/app/src/main/cpp/native_sim.cpp	19, 116, 203, 206, 219
android-sim/app/src/main/cpp/rust_control_jni.cpp	10
android-sim/app/src/main/cpp/visual_snapshot.h	14
android-sim/app/src/main/java/com/microduck/sim/LanControlServer.java	42
android-sim/app/src/main/java/com/microduck/sim/MainActivity.java	39, 171, 205, 284, 433, 571
android-sim/app/src/main/java/com/microduck/sim/ModelAssets.java	16, 41
android-sim/app/src/main/java/com/microduck/sim/NativeSim.java	5
android-sim/app/src/main/java/com/microduck/sim/PolicyRunner.java	55, 59, 90, 97
android-sim/app/src/main/java/com/microduck/sim/RemoteLease.java	5
android-sim/app/src/main/java/com/microduck/sim/RemoteMailbox.java	1
android-sim/app/src/main/java/com/microduck/sim/ReportDispatcher.java	13
android-sim/app/src/main/java/com/microduck/sim/RouladeWindow.java	5
android-sim/app/src/main/java/com/microduck/sim/RtcSession.java	9
android-sim/app/src/main/java/com/microduck/sim/RtcVideo.java	93
android-sim/app/src/main/java/com/microduck/sim/SceneFrame.java	26
android-sim/app/src/main/java/com/microduck/sim/SceneRenderer.java	148, 183
android-sim/app/src/main/java/com/microduck/sim/SimIoBody.java	18
android-sim/app/src/main/java/com/microduck/sim/SimIoServer.java	50
android-sim/app/src/main/java/com/microduck/sim/VideoCapture.java	34, 49, 72
android-sim/build.gradle	3
android-sim/gradle/wrapper/gradle-wrapper.properties	3
android-sim/patches/mujoco-3.10.0-ndk25.patch	1
android-sim/rust/control-core/Cargo.toml	6, 14
android-sim/rust/control-core/build.rs	8
android-sim/rust/control-core/src/lib.rs	4, 6, 62, 105
android-sim/rust/control-core/src/policy.rs	18
android-sim/scripts/android_robotd.py	29
android-sim/scripts/build_rust_core.py	10, 28
android-sim/scripts/check_source.py	1, 105
android-sim/scripts/head_recovery_rehearsal.py	1
android-sim/scripts/prepare_assets.py	3, 9, 36
android-sim/settings.gradle	3
android-sim/verification/device-M71-roll-acceptance.json	1
android-sim/verification/device-M71-roll-log-samples.json	1
android-sim/verification/host-M71-head-recovery.json	1
android-sim/verification/m10-android-rust-checks.json	1

文件（工作区相对路径）	引用行号
android-sim/verification/m10-checks.json	1
android-sim/verification/m10-embedded-rust-checks.json	1
android-sim/verification/m8-video-dependencies.json	1
android-sim/verification/m9-bam-checks.json	1
android-sim/verification/user-M10-feedback.json	1
android-sim/verification/user-M9-simio-feedback.json	1